SA SisAcad

Documentação Técnica do Projeto

Sistema Acadêmico de Biblioteca e Achados e Perdidos — planejamento, requisitos, modelagem, arquitetura, implementação e testes, com explicações.

Programação para Internet · 2º bimestre

Java · Spring Boot · MySQL · HTML/CSS/JS

Apresentação: 19/11

Sumário

1 Planejamento 2
2 Requisitos funcionais e não funcionais 3
2.4 Regras de negócio 4
3 Casos de uso 5
4 Diagrama de classes 6
4.2 Diagramas de estados e conceitos de OO 7
5 Banco de dados (modelo ER) 8
6 Arquitetura em camadas 9
6.2 Fluxo de uma requisição 10
7 API REST 11
8 Frontend 12
9 Testes 13
10 Glossário 14

Como o projeto foi construído

Planejar
Problema, público, requisitos e escopo

Modelar
Casos de uso, classes, estados e banco

Implementar
Backend Spring Boot e frontend HTML/CSS/JS

Testar
Testes automáticos, manuais e evidências

Como usar este documento

Cada capítulo traz o artefato pedido (tabela ou diagrama), uma explicação de por que ele é assim e, nas caixas verdes, uma frase pronta para usar na apresentação. Os nomes nos diagramas são exatamente os mesmos do código-fonte.

ETAPA 1 · PLANEJAMENTO

1. Planejamento

O planejamento define qual problema resolvemos, para quem, o que entra no sistema e o que fica de fora, garantindo que a entrega caiba no prazo de 19/11.

Problema

O controle do acervo da biblioteca e dos objetos perdidos é feito em papel ou planilhas. Não se sabe com rapidez se um livro está disponível, com quem está e desde quando, nem se um objeto encontrado já foi devolvido ao dono.

Objetivo

Centralizar os dois controles em uma aplicação web com banco de dados, que registre empréstimos com histórico e acompanhe cada objeto encontrado até a devolução.

Público-alvo

Grupo	Papel no sistema	O que ganha
Operadores — funcionários da biblioteca e da secretaria	Usam a interface administrativa (único ator do sistema)	Cadastro e consulta rápidos; fim das planilhas
Beneficiados — alunos e servidores	Cadastrados como usuários dos empréstimos; não acessam o sistema	Atendimento mais rápido; objetos perdidos localizados com facilidade

Escopo

Dentro do escopo

- Módulo Biblioteca: livros, empréstimos, devoluções e histórico
- Módulo Achados e Perdidos: registro, status e devolução
- Painel com indicadores
- API REST, banco MySQL (ou H2), validações e testes

Fora do escopo (evoluções futuras)

- Login e perfis de acesso
- Multa por atraso e reserva de livros
- Notificação por e-mail
- Fotos dos objetos encontrados

Cronograma



Na apresentação

"Nosso sistema substitui as planilhas da biblioteca e do achados e perdidos. Quem usa é o funcionário; quem ganha é o aluno, que é atendido mais rápido. Deixamos login e multas fora do escopo para entregar no prazo."

ETAPA 1 · PLANEJAMENTO

2. Requisitos

Requisito funcional diz **o que** o sistema faz; não funcional diz **como** ou com quais restrições. Os três primeiros RFs resumem o sistema; os demais os detalham.

2.1 Requisitos funcionais principais

ID	Requisito
RF01	**Gerenciar livros:** cadastrar, listar, pesquisar por título ou autor, editar e excluir
RF02	**Controlar empréstimos:** registrar empréstimo e devolução, mantendo histórico e disponibilidade
RF03	**Controlar achados e perdidos:** registrar objetos com descrição livre, atualizar status e registrar a devolução ao dono

2.2 Requisitos funcionais detalhados

ID	Requisito	Módulo	Origem
RF04	Cadastrar livro (título, autor, ISBN, categoria)	Biblioteca	RF01
RF05	Consultar livros por título ou autor e por disponibilidade	Biblioteca	RF01
RF06	Editar dados do livro e excluir livro sem empréstimos	Biblioteca	RF01
RF07	Cadastrar usuários (nome, matrícula, e-mail)	Biblioteca	RF02
RF08	Registrar empréstimo (livro, usuário, data)	Biblioteca	RF02
RF09	Registrar devolução de livro	Biblioteca	RF02
RF10	Exibir o histórico de empréstimos, filtrando ativos e devolvidos	Biblioteca	RF02
RF11	Registrar objeto encontrado (nome, descrição, local, data)	Achados	RF03
RF12	Listar e pesquisar objetos por nome, descrição ou local	Achados	RF03
RF13	Editar dados do objeto e atualizar status (Encontrado ↔ Em estoque)	Achados	RF03
RF14	Registrar devolução do objeto informando quem retirou	Achados	RF03
RF15	Exibir painel com indicadores dos dois módulos	Geral	—

2.3 Requisitos não funcionais

ID	Requisito	Como foi atendido
RNF01	Backend em Java com Spring Boot e Maven	Java 17, Spring Boot 3.3, `pom.xml`
RNF02	Persistência em banco relacional	MySQL 8 via Spring Data JPA; H2 para demonstração
RNF03	Integração por API REST com JSON	Controllers REST + `fetch()` no frontend
RNF04	Interface web responsiva sem frameworks	HTML5, CSS3 (grid, media queries) e JavaScript puro
RNF05	Arquitetura em camadas	Pacotes controller, service, repository, model
RNF06	Validação de dados e mensagens de erro claras	Validação no navegador e no backend (`@NotBlank`)
RNF07	Integridade dos dados	PK, FK, UNIQUE, CHECK e `@Transactional`
RNF08	Fácil de executar em outro computador	Um comando ou F5 no VS Code; modo H2 sem instalar banco

ETAPA 1 · PLANEJAMENTO

2.4 Regras de negócio

Regras de negócio são as condições que o sistema precisa garantir sempre, independentemente da tela. Por isso elas ficam no backend, na camada **Service**, e não no JavaScript.

ID	Regra	Onde está no código
RN01	Todo livro novo inicia como **DISPONÍVEL**	`LivroService.criar()`
RN02	O ISBN não pode se repetir	`LivroService.criar()` + UNIQUE no banco
RN03	Ao registrar empréstimo, o livro passa para **EMPRESTADO**	`EmprestimoService.registrar()`
RN04	Não é permitido emprestar livro já emprestado	`EmprestimoService.registrar()`
RN05	Na devolução, empréstimo vira **DEVOLVIDO** e livro volta a **DISPONÍVEL**	`EmprestimoService.registrarDevolucao()`
RN06	Empréstimos nunca são apagados (histórico)	Não existe rota DELETE de empréstimo
RN07	Livro com empréstimos registrados não pode ser excluído	`LivroService.excluir()` + FK RESTRICT
RN08	A data do empréstimo não pode ser futura	`EmprestimoService.registrar()`
RN09	Todo objeto novo inicia como **ENCONTRADO**	`ObjetoEncontradoService.registrar()`
RN10	Objeto pode alternar entre ENCONTRADO e **EM_ESTOQUE**	`ObjetoEncontradoService.atualizarStatus()`
RN11	Objeto só vira **DEVOLVIDO** pela devolução, informando quem retirou	`ObjetoEncontradoService.registrarDevolucao()`
RN12	Objeto devolvido não muda mais de status	`ObjetoEncontradoService.atualizarStatus()`

Como uma regra é aplicada

```java
// EmprestimoService.java — RN04 e RN03
if (!livro.isDisponivel()) {
    throw new RegraNegocioException("O livro já está emprestado.");   // vira HTTP 409
}
livro.setStatus(StatusLivro.EMPRESTADO);
livroRepository.save(livro);
return emprestimoRepository.save(new Emprestimo(usuario, livro, data));
```

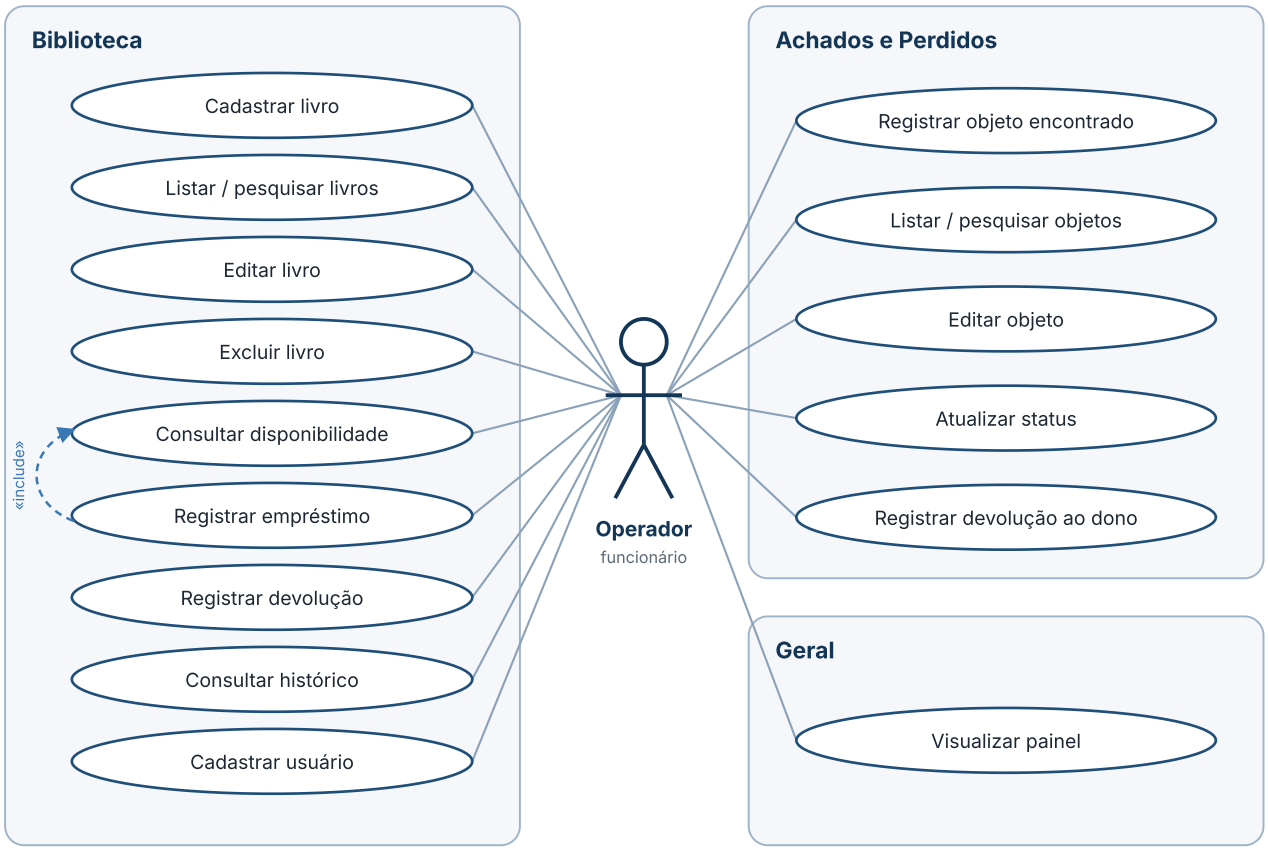
Por que no backend e não na tela?
Se a regra estivesse só no JavaScript, qualquer pessoa poderia enviar uma requisição direto para a API (pelo Postman, por exemplo) e burlar a regra. No backend ela vale para qualquer cliente. A tela apenas mostra a mensagem que o backend devolve.

Na apresentação
"As regras ficam nos services. Se alguém tentar emprestar um livro já emprestado, o service lança uma exceção, que vira o código HTTP 409 e aparece como aviso vermelho na tela."

ETAPA 2 · MODELAGEM

3. Diagrama de casos de uso

Mostra **quem** usa o sistema e **o que** pode fazer com ele. Há um único ator, o Operador, que usa os dois módulos.



Descrição do caso de uso: Registrar empréstimo

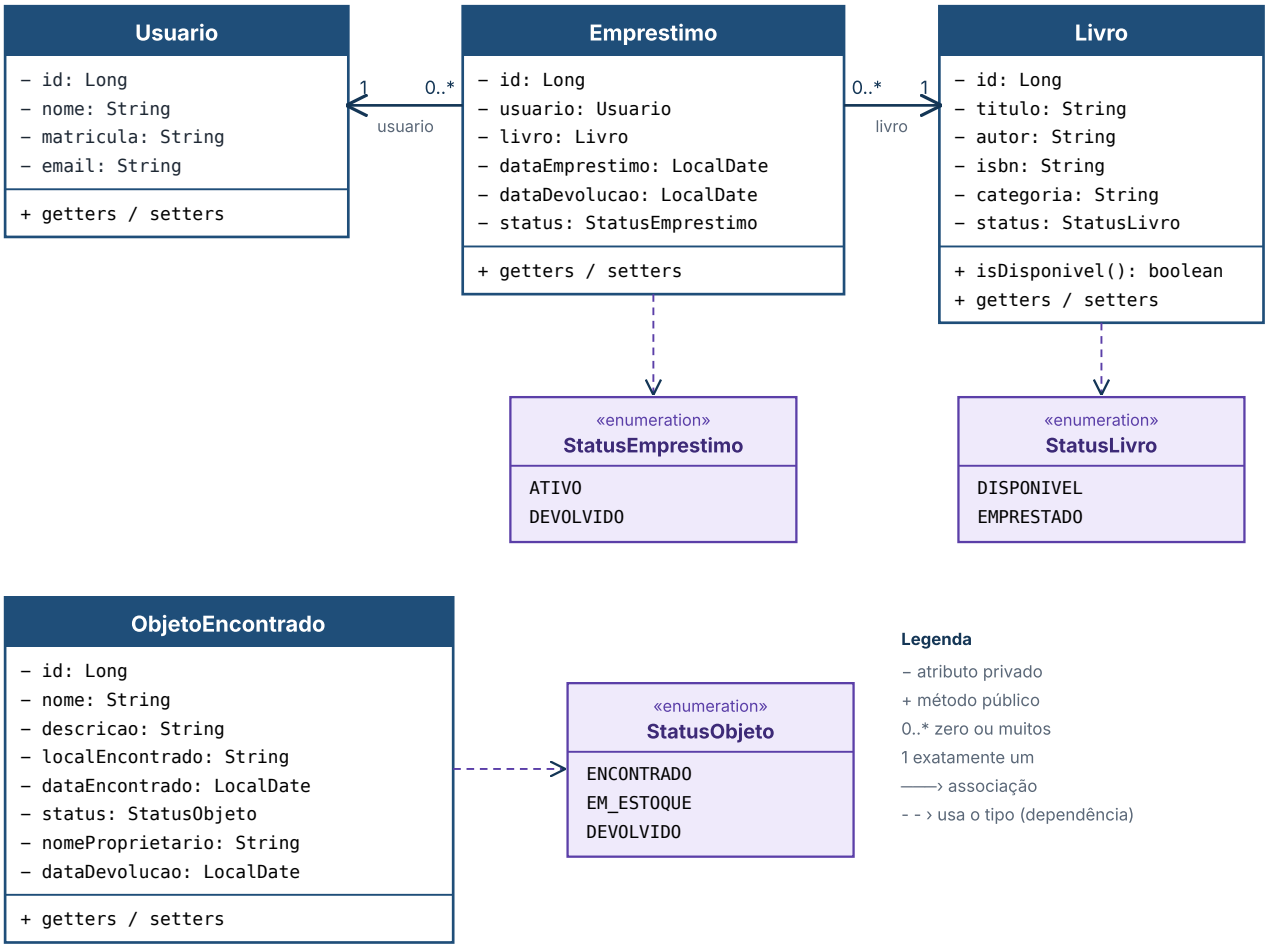
Ator	Operador
Pré-condição	Livro e usuário cadastrados
Fluxo principal	1. Abre Empréstimos · 2. Seleciona um livro (só aparecem os disponíveis) e um usuário · 3. Informa a data · 4. Confirma · 5. O sistema grava o empréstimo como ATIVO e muda o livro para EMPRESTADO
Fluxo alternativo	5a. Livro já emprestado: o sistema exibe "O livro … já está emprestado" e nada é gravado
Pós-condição	Empréstimo no histórico; livro indisponível para novos empréstimos

O que significa «include»?
Registrar empréstimo sempre inclui consultar a disponibilidade: o sistema verifica se o livro está livre antes de gravar. Por isso só livros disponíveis aparecem na lista da tela.

ETAPA 2 · MODELAGEM

4. Diagrama de classes

As classes representam os conceitos reais do problema. Os nomes, atributos e tipos são idênticos aos do pacote `model` do projeto.



Como ler o diagrama

- **Emprestimo liga Usuario e Livro:** cada empréstimo tem exatamente 1 usuário e 1 livro; um livro pode ter 0 ou muitos empréstimos ao longo do tempo. É isso que forma o histórico.
- **Tipos adequados:** datas em `LocalDate` (permite comparar e validar datas futuras), ids em `Long`, status em `enum`, que só aceita os valores previstos.
- **Métodos:** `isDisponivel()` encapsula a pergunta "o livro está livre?". As operações (registrar, devolver) ficam nas classes de serviço, mostradas no capítulo 6.
- **ObjetoEncontrado é independente:** quem perde um objeto normalmente não é cadastrado, então guardamos apenas o nome de quem retirou.

ETAPA 2 · MODELAGEM

4.2 Diagramas de estados

Mostram por quais status cada objeto passa e qual ação provoca cada mudança. São a forma visual das regras de negócio RN01 a RN12.



● estado inicial · ◉ estado final · cada seta é uma ação do operador. O objeto pode ser devolvido estando Encontrado ou Em estoque; depois de Devolvido não muda mais.

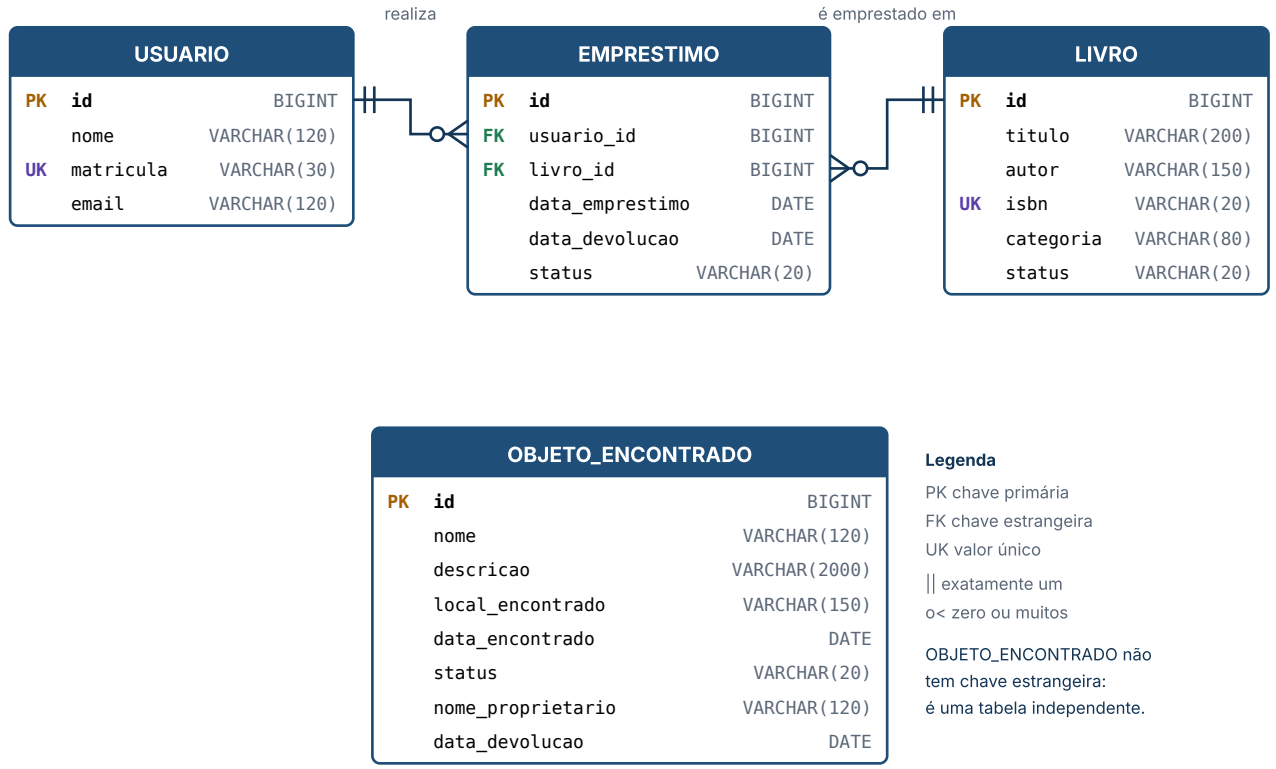
4.3 Conceitos de orientação a objetos no projeto

Conceito	Onde aparece
Classe e objeto	`Livro` é a classe; cada livro cadastrado é um objeto dessa classe
Encapsulamento	Atributos `private` com getters e setters; o status do livro só muda pelos métodos do service
Associação	`Emprestimo` possui um `Usuario` e um `Livro` (`@ManyToOne`)
Enumeração	`StatusLivro`, `StatusEmprestimo`, `StatusObjeto` impedem valores inválidos
Herança	`RegraNegocioException` e `RecursoNaoEncontradoException` herdam de `RuntimeException`
Interface	Os repositories são interfaces que estendem `JpaRepository`; o Spring gera a implementação
Responsabilidade única	Cada classe tem um papel: Controller recebe, Service decide, Repository grava
Injeção de dependência	O Spring entrega os repositories prontos no construtor dos services

ETAPA 2 · MODELAGEM

5. Banco de dados — modelo entidade-relacionamento

Quatro tabelas. **EMPRESTIMO** liga USUARIO a LIVRO e guarda o histórico; **OBJETO_ENCONTRADO** é independente.



Restrições que protegem os dados

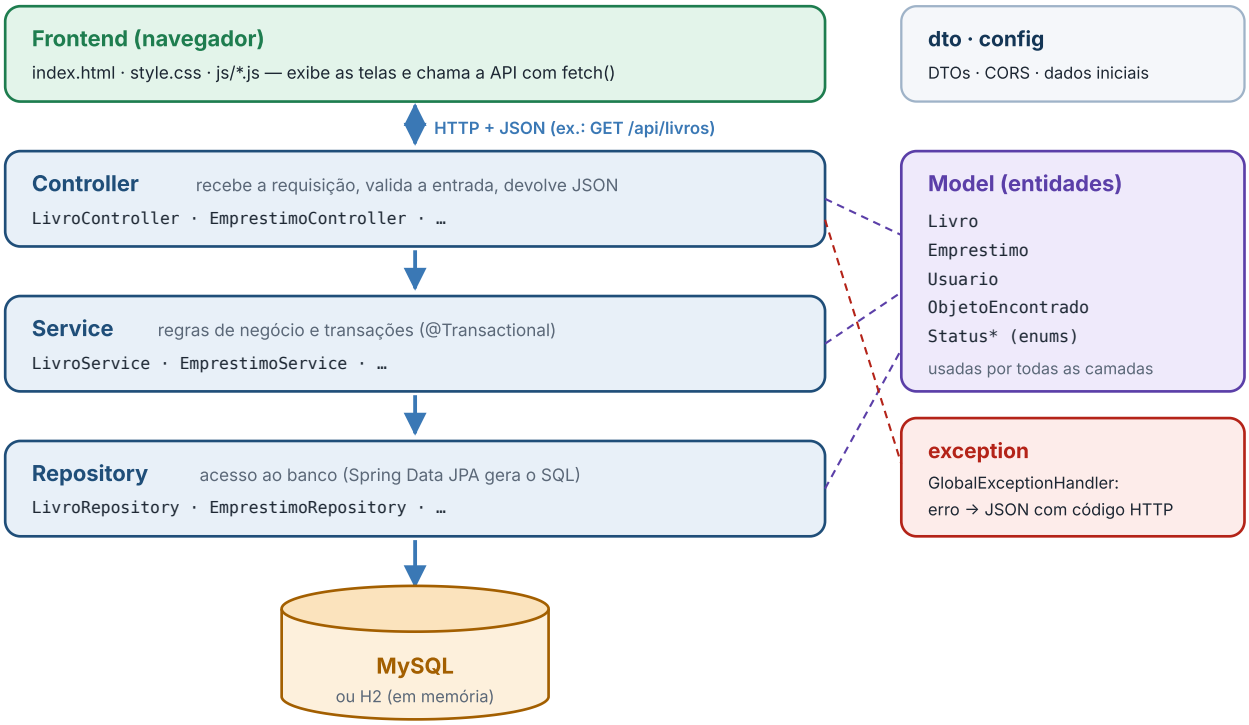
Restrição	Efeito
PRIMARY KEY (id)	Cada registro tem um identificador único, gerado automaticamente (`AUTO_INCREMENT`)
FOREIGN KEY usuario_id, livro_id	Um empréstimo só pode apontar para usuário e livro que existem
ON DELETE RESTRICT	O banco recusa apagar livro ou usuário que tenha empréstimos — reforça a RN07
UNIQUE isbn, matricula	Impede livros com o mesmo ISBN e usuários com a mesma matrícula
CHECK status	Só aceita os valores dos enums (ex.: DISPONIVEL ou EMPRESTADO)

Quem cria as tabelas?
O Hibernate cria e ajusta as tabelas a partir das classes `@Entity` ao iniciar a aplicação. O arquivo `database/script.sql` documenta o mesmo modelo para quem quiser criar pelo MySQL Workbench.

ETAPA 3 · IMPLEMENTAÇÃO

6. Arquitetura em camadas

Uma única aplicação Spring Boot dividida em camadas. Cada camada só conversa com a de baixo, e cada uma tem uma responsabilidade clara.



Organização dos pacotes

Pacote	Conteúdo	Classes
`controller`	Endpoints REST de livros, empréstimos, usuários, objetos e painel	5
`service`	Regras de negócio de cada módulo e cálculo do painel	5
`repository`	Interfaces Spring Data JPA de acesso ao banco	4
`model`	Entidades JPA e enums de status	7
`dto`	Objetos de entrada e saída da API (ex.: `EmprestimoRequest`)	5
`exception`	Exceções de negócio e tratamento global de erros	3
`config`	Liberação de CORS e dados de exemplo da primeira execução	2

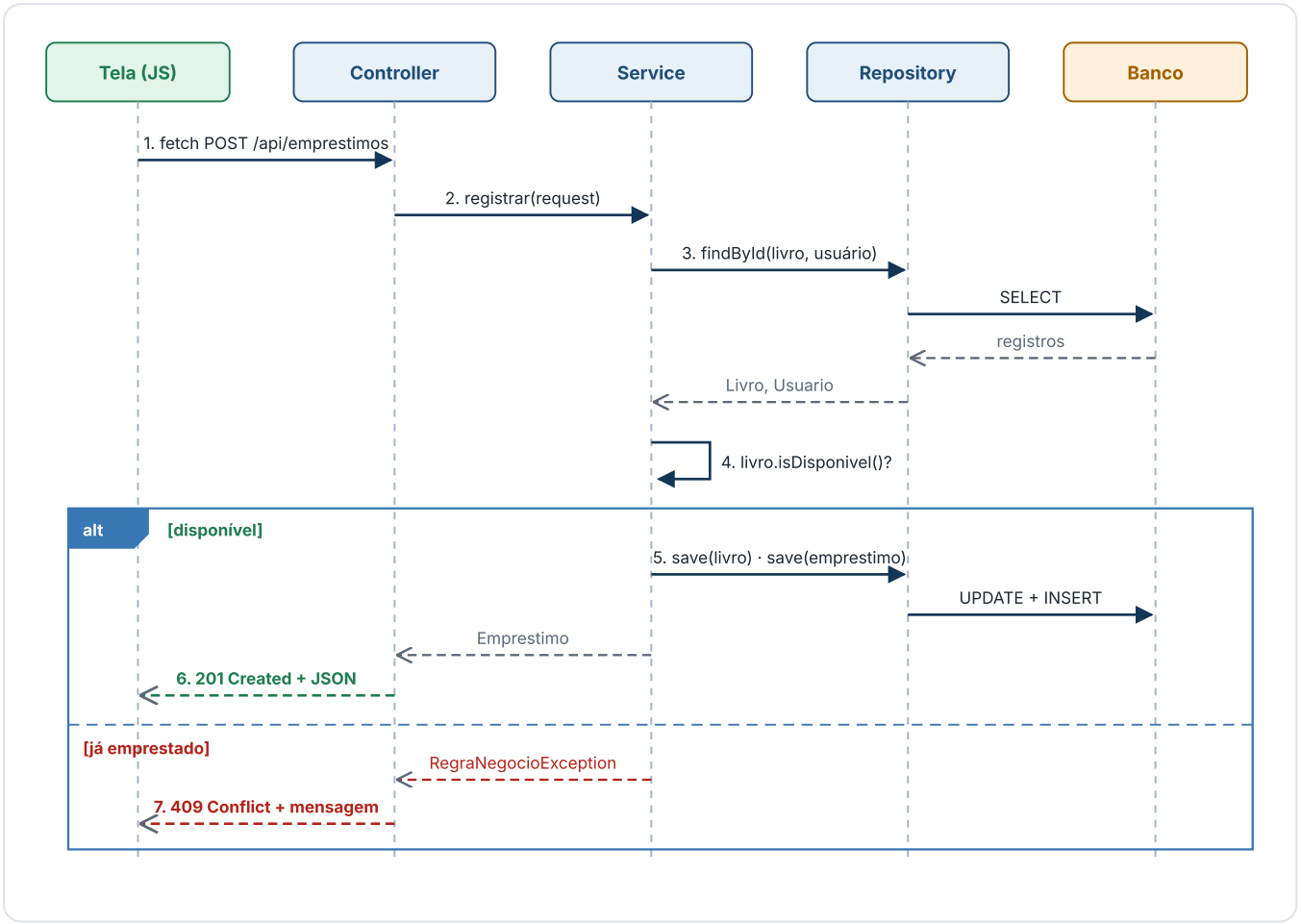
Na apresentação

"Usamos arquitetura em camadas: o controller só recebe e responde, o service concentra as regras e o repository conversa com o banco. Assim cada parte pode ser entendida e testada separadamente."

ETAPA 3 · IMPLEMENTAÇÃO

6.2 Fluxo de uma requisição

Diagrama de sequência do caso "Registrar empréstimo": o caminho de um clique na tela até o banco de dados e de volta.



Passo a passo em palavras

1. **Tela:** o operador escolhe livro e usuário; o JavaScript envia `POST /api/emprestimos` com `{"livroId":1,"usuarioId":2}`.
2. **Controller:** `EmprestimoController` valida os campos e chama o service.
3. **Service:** busca livro e usuário; se não existirem, responde 404.
4. **Regra:** verifica se o livro está disponível (RN04).
5. **Gravação:** muda o livro para EMPRESTADO e salva o empréstimo na mesma transação.
6. **Sucesso:** a API responde 201 com o empréstimo em JSON; a tela mostra o aviso verde e recarrega a lista.
7. **Falha:** se o livro já estiver emprestado, a exceção vira 409 e a tela mostra o aviso vermelho com a mensagem.

ETAPA 3 · IMPLEMENTAÇÃO

7. API REST

Cada recurso tem um endereço (`/api/livros`) e o verbo HTTP indica a ação. Todas as respostas são em JSON. Base: `http://localhost:8080/api`

Verbo	Endpoint	O que faz
GET	`/livros?busca=&status=`	Lista livros; filtra por título/autor e status
GET	`/livros/{id}`	Detalha um livro
GET	`/livros/{id}/disponibilidade`	Informa se o livro está disponível
POST	`/livros`	Cadastra livro (sempre DISPONÍVEL)
PUT	`/livros/{id}`	Edita dados cadastrais
DELETE	`/livros/{id}`	Exclui (bloqueado se houver empréstimos)
GET	`/emprestimos?status=`	Histórico; filtra ativos ou devolvidos
POST	`/emprestimos`	Registra empréstimo
PUT	`/emprestimos/{id}/devolucao`	Registra devolução
GET	`/usuarios`	Lista usuários
POST	`/usuarios`	Cadastra usuário
GET	`/objetos?busca=&status=`	Lista e pesquisa objetos
POST	`/objetos`	Registra objeto (sempre ENCONTRADO)
PUT	`/objetos/{id}`	Edita dados do objeto
PATCH	`/objetos/{id}/status`	Alterna Encontrado ↔ Em estoque
PUT	`/objetos/{id}/devolucao`	Devolve ao dono (informa quem retirou)
DELETE	`/objetos/{id}`	Exclui registro
GET	`/dashboard`	Números do painel

Exemplo: GET /api/livros

```
[
  {
    "id": 1,
    "titulo": "Código Limpo",
    "autor": "Robert C. Martin",
    "isbn": "9788576082675",
    "categoria": "Computação",
    "status": "DISPONIVEL",
    "disponivel": true
  }
]
```

Códigos de resposta

200	OK — consulta ou alteração feita
201	Criado — novo registro
204	Sem conteúdo — exclusão feita
400	Dados inválidos (campo obrigatório vazio)
404	Registro não encontrado
409	Regra de negócio violada
500	Erro inesperado no servidor

ETAPA 3 · IMPLEMENTAÇÃO

8. Frontend

HTML, CSS e JavaScript puros, servidos pelo próprio Spring Boot (pasta `src/main/resources/static`). Uma única página com quatro telas, trocadas pelo menu.

Tela	O que oferece	Endpoints usados
Painel	Totais de livros (disponíveis e emprestados) e de objetos (em aberto e devolvidos)	`GET /dashboard`
Biblioteca	Formulário de cadastro/edição, busca por título ou autor, filtro por status, badges coloridos	`GET/POST/PUT/DELETE /livros`
Empréstimos	Seleção de livro disponível e usuário, data, botão de devolução, histórico, cadastro de usuário	`/emprestimos` · `/usuarios` · `/livros?status=DISPONIVEL`
Achados e Perdidos	Cadastro com descrição livre, busca, troca de status, devolução ao dono com janela de confirmação	`/objetos` · `/objetos/{id}/status` · `/devolucao`

Arquivos JavaScript

Arquivo	Responsabilidade
`api.js`	Função `requisicao()` com `fetch()`, avisos na tela, formatação de datas e proteção contra HTML injetado
`painel.js`	Carrega os indicadores
`livros.js` · `emprestimos.js` · `objetos.js`	Lógica de cada tela: carregar lista, salvar, editar, excluir
`app.js`	Navegação entre as telas e inicialização

Como o fetch() funciona no projeto

```
// api.js — usada por todas as telas
async function requisicao(caminho, metodo = 'GET', corpo = null) {
    const opcoes = { method: metodo, headers: {} };
    if (corpo) {
        opcoes.headers['Content-Type'] = 'application/json';
        opcoes.body = JSON.stringify(corpo);          // objeto JS → texto JSON
    }
    const resposta = await fetch(API_URL + caminho, opcoes);  // chama o backend
    const dados = await resposta.json();                  // texto JSON → objeto JS
    if (!resposta.ok) throw new Error(dados.mensagem);        // 400, 404, 409...
    return dados;
}

// livros.js — exemplo de uso
livrosCarregados = await requisicao('/livros?busca=machado');
```

Ver o fetch acontecendo

No navegador, aperte F12 → aba Network (Rede) → filtro Fetch/XHR. Cada ação na tela mostra a chamada para `/api/...`, o verbo, o código de resposta e o JSON recebido.

ETAPA 4 · TESTES

9. Testes

Três níveis de teste: automáticos de unidade (regras), automático de integração (API real) e manuais pelas telas e pelo Postman.

9.1 Testes automáticos — JUnit 5 e Mockito (13 testes)

Classe de teste	O que verifica	Regras
`LivroServiceTest`	Livro novo entra DISPONÍVEL · ISBN duplicado é recusado · livro com histórico não pode ser excluído	RN01 RN02 RN07
`EmprestimoServiceTest`	Empréstimo muda livro para EMPRESTADO · não empresta livro emprestado · devolução libera o livro · não devolve duas vezes	RN03 RN04 RN05
`ObjetoEncontradoServiceTest`	Objeto novo entra ENCONTRADO · DEVOLVIDO só pela devolução · devolução registra dono e data	RN09 RN11
`ApiIntegracaoTest`	GET /api/livros responde 200 com JSON · emprestar duas vezes responde 409 · livro inexistente responde 404	C3 C4

Como rodar: VS Code → ícone de frasco (Testing) → Run All Tests. Os testes de unidade usam **Mockito** para simular os repositórios, sem banco; o de integração sobe a aplicação com o banco H2.

9.2 Casos de teste manuais

ID	Cenário	Resultado esperado
CT01	Cadastrar livro com todos os campos	Livro aparece na lista como Disponível
CT02	Cadastrar sem título	Campo fica vermelho; nada é enviado
CT03	Cadastrar ISBN repetido	Aviso "Já existe um livro com o ISBN..." (409)
CT04	Pesquisar parte do nome do autor	Lista filtrada sem recarregar a página
CT06	Registrar empréstimo	Empréstimo Ativo; livro Emprestado e fora da lista de disponíveis
CT07	Emprestar livro já emprestado (Postman)	409 "já está emprestado"; nada gravado
CT09	Registrar devolução	Empréstimo Devolvido com data; livro Disponível
CT11	Excluir livro que já foi emprestado	Exclusão bloqueada; histórico intacto
CT13	Registrar objeto com descrição detalhada	Card com status Encontrado
CT14	Mover objeto para estoque	Status Em estoque
CT15	Devolver objeto informando o dono	Status Devolvido; card mostra quem retirou e quando
CT18	Abrir o painel após as operações	Números coerentes com o que foi feito
CT19	GET /api/livros/9999	404 "Livro 9999 não encontrado."

Lista completa (20 casos) em `docs/documentacao-tecnica.md`. Prints de execução em `docs/evidencias/`.

REFERÊNCIA

10. Glossário

Os termos técnicos usados no projeto, explicados em uma frase cada.

Termo	Significado
Java	Linguagem orientada a objetos usada no backend.
JDK	Kit de desenvolvimento Java: compilador e máquina virtual para rodar o código.
Spring Boot	Framework que cria aplicações Java prontas para rodar, com servidor web embutido (Tomcat).
Maven	Ferramenta que baixa as bibliotecas declaradas no `pom.xml` e compila o projeto.
API REST	Conjunto de endereços HTTP que trocam dados; cada recurso tem uma URL e o verbo indica a ação.
HTTP (GET, POST, PUT, PATCH, DELETE)	Protocolo da web; os verbos significam consultar, criar, substituir, alterar parte e excluir.
JSON	Formato de texto para trocar dados entre frontend e backend: `{"titulo": "Dom Casmurro"}`.
fetch()	Função do JavaScript que faz requisições HTTP sem recarregar a página.
async / await	Forma de esperar a resposta do servidor sem travar a tela.
JPA / Hibernate	Mapeiam classes Java para tabelas do banco e geram o SQL automaticamente.
Spring Data JPA	Cria consultas a partir do nome do método, ex.: `findByTituloContainingIgnoreCase`.
Entidade (@Entity)	Classe que vira tabela no banco.
Controller / Service / Repository	Camadas que recebem a requisição, aplicam as regras e acessam o banco.
DTO	Objeto só para transportar dados de uma requisição, sem expor a entidade inteira.
@Transactional	Garante que várias gravações aconteçam juntas ou nenhuma aconteça.
Bean Validation	Anotações como `@NotBlank` que validam os dados antes de gravar.
Chave primária / estrangeira	Identificador único do registro / campo que aponta para registro de outra tabela.
H2	Banco de dados em memória usado para testes e demonstração sem instalar MySQL.
CORS	Permissão para uma página de outro endereço chamar a API.
JUnit / Mockito	Ferramentas de teste automático / de simulação de dependências nos testes.
UML	Linguagem visual para modelar sistemas: casos de uso, classes, estados, sequência.
Postman	Programa para testar a API enviando requisições manualmente.